

Website TSX Handoff

From Cory → Shub

June 2026 · Confidential

This document covers everything Shub needs to integrate the augle.com marketing website TSX files into the Next.js project. It documents every delivered file, all shared component interfaces, key implementation notes, wiring requirements, and the pages still to be built.

1. Tech stack & project setup

Framework

Framework	Next.js 14+ App Router
Language	TypeScript
Styling	Tailwind CSS with Augle design tokens
Components	shadcn/ui
Utility	cn() from clsx/tailwind-merge

Required: tailwind.config.ts

All TSX files reference Augle design tokens as Tailwind classes. The config must define these tokens — a corrected tailwind.config.ts was previously produced. Key tokens:

```
colors: {
  sand:      '#E8E4D9',    // bg-sand
  surface:   '#F4F1EB',    // bg-surface
  dark:      '#1E1C1A',    // bg-dark / text-dark
  terracotta: '#C15F3C',   // text-terracotta / bg-terracotta
  border:    '#C8C4BA',    // border-border
  muted:     '#B0ADA5',    // text-muted
  body:      '#5A5550',    // text-body
},
fontFamily: {
  serif: ['Libre Baskerville', 'serif'],
  mono:  ['IBM Plex Mono', 'monospace'],
  sans:  ['IBM Plex Sans', 'sans-serif'],
},
```

Google Fonts

Add to app/layout.tsx head — all three families required:

```
Libre Baskerville — headings / serif
IBM Plex Sans    — body text (19px base)
IBM Plex Mono    — labels, eyebrows, monospace UI
```

Shared components location

All shared components live at components/ at the project root (not inside app/):

```
components/
  Nav.tsx          // sticky nav, mobile hamburger, activeRoute prop
  Footer.tsx       // dark bg, 5-col grid, both logo SVG paths
  WaffleGrid.tsx   // 5×20 confidence grid, WaffleAgent type
  Breadcrumb.tsx   // items[] prop, terracotta active terminal
```

Component interfaces

Nav.tsx

```
interface NavProps { activeRoute?: string }
// activeRoute matches against href — e.g. '/solutions', '/outcomes'
// Pass the top-level route, not the full path
// e.g. /solutions/universities → activeRoute='/solutions'
```

Breadcrumb.tsx

```
interface BreadcrumbItem { label: string; href?: string }
interface BreadcrumbProps { items: BreadcrumbItem[] }
// Last item (no href) renders as terracotta active terminal
```

WaffleGrid.tsx

```
export interface WaffleAgent {
  name: string
  pct: number    // 0-100
  color: string  // hex — '#C15F3C' | '#8AAA9A' | '#A05040'
}
interface WaffleGridProps {
  agents: WaffleAgent[]
  isActive?: boolean // triggers animation when true
}
```

2. Delivered zips — complete inventory

Six zips delivered. Unzip each and drop the app/ folder contents into the Next.js app/ directory. The components/ folder in the Core zip goes to the project root.

augle_core_pages_tsx.zip

Contains shared components (components/) + core pages (app/)

File / Component	Route	Notes
components/Nav.tsx	—	Sticky nav, mobile hamburger, activeRoute prop
components/Footer.tsx	—	Dark bg, 5-col grid, both logo SVG paths required
components/WaffleGrid.tsx	—	WaffleAgent type, isActive animation
components/Breadcrumb.tsx	—	items[] prop, terracotta active terminal
app/page.tsx	/	'use client' — animated waffle, cycling questions
app/pricing/page.tsx	/pricing	'use client' — email state, tier cards, FAQ
app/about/page.tsx	/about	Server component — v2 layout, no team section
app/research/page.tsx	/research	Server — 7 papers, Zenodo/SSRN links
app/how-it-works/page.tsx	/how-it-works	Server — hub with dispatch strip
app/how-it-works/agents/page.tsx	/how-it-works/agents	Server — agent spec table
app/how-it-works/phases/page.tsx	/how-it-works/phases	Server — phase detail
app/how-it-works/guardian/page.tsx	/how-it-works/guardian	Server — flag taxonomy, SVS
app/how-it-works/scoring/page.tsx	/how-it-works/scoring	Server — grades, propagation
app/index/page.tsx	/index	Server — Index overview, domain table, Brier chart
app/outcomes/page.tsx	/outcomes	'use client' — filter, mode toggle, sidebar

augle_outcomes_index_tsx.zip

File / Component	Route	Notes
<code>app/outcomes/hub/page.tsx</code>	<code>/outcomes</code>	Static marketing hub — calibration panel, session cards
<code>app/outcomes/browser/page.tsx</code>	<code>/outcomes/browser</code>	'use client' — running/resolved toggle, category filter, sidebar
<code>app/outcomes/[id]/page.tsx</code>	<code>/outcomes/[id]</code>	'use client' — single template, conditional on session.mode
<code>app/index/explorer/page.tsx</code>	<code>/index/explorer</code>	'use client' — filter panel, 31-session dataset, table view
<code>app/index/heatmap/page.tsx</code>	<code>/index/heatmap</code>	'use client' — clickable cells, count/% toggle, sidebar
<code>app/index/methodology/page.tsx</code>	<code>/index/methodology</code>	Server — full methodology prose

augle_solutions_tsx.zip

Shared layout: `SolutionLayout.tsx` + `verticalData.ts` live at `app/solutions/` level

File / Component	Route	Notes
<code>app/solutions/page.tsx</code>	<code>/solutions</code>	11-card index grid with jump links
<code>app/solutions/SolutionLayout.tsx</code>	—	Shared layout template — all 11 verticals use this
<code>app/solutions/verticalData.ts</code>	—	Data for Think Tanks, Enterprise, Healthcare, Government, Financial Services, Media
<code>app/solutions/universities/page.tsx</code>	<code>/solutions/universities</code>	Self-contained data + <code>SolutionLayout</code>
<code>app/solutions/research-labs/page.tsx</code>	<code>/solutions/research-labs</code>	Self-contained data + <code>SolutionLayout</code>
<code>app/solutions/policy/page.tsx</code>	<code>/solutions/policy</code>	Self-contained data + <code>SolutionLayout</code>
<code>app/solutions/law-firms/page.tsx</code>	<code>/solutions/law-firms</code>	Self-contained data + <code>SolutionLayout</code>

File / Component	Route	Notes
<code>app/solutions/venture-capital/page.tsx</code>	<code>/solutions/venture-capital</code>	Self-contained data + SolutionLayout
<code>think-tanks through media (6 files)</code>	<code>/solutions/[vertical]</code>	Thin — import from verticalData.ts

augle_use_cases_tsx.zip

Shared layout: `UseCaseHubLayout.tsx` + `hubData.ts` live at `app/use-cases/` level

File / Component	Route	Notes
<code>app/use-cases/page.tsx</code>	<code>/use-cases</code>	Server — 11-vertical index, cross-vertical patterns
<code>app/use-cases/UseCaseHubLayout.tsx</code>	—	'use client' — accordion sessions, defaultOpen first
<code>app/use-cases/hubData.ts</code>	—	All session data for 10 hubs (Research Labs → Media)
<code>app/use-cases/universities/page.tsx</code>	<code>/use-cases/universities</code>	Self-contained data + UseCaseHubLayout
<code>research-labs through media (10 files)</code>	<code>/use-cases/[vertical]</code>	Thin — import from hubData.ts

augle_misc_tsx.zip

File / Component	Route	Notes
<code>app/contact/page.tsx</code>	<code>/contact</code>	'use client' — 4 channel cards, form with useState, team section
<code>app/glossary/page.tsx</code>	<code>/glossary</code>	'use client' — 20 terms, live search filter, sidebar jump links

augle_legal_tsx.zip

File / Component	Route	Notes
<code>app/privacy/page.tsx</code>	<code>/privacy</code>	Server — sticky sidebar TOC, 11 sections, cross-links
<code>app/terms/page.tsx</code>	<code>/terms</code>	Server — sticky sidebar TOC, credit table, refund schedule
<code>app/terms-of-service/page.tsx</code>	<code>/terms-of-service</code>	Server — sticky sidebar TOC, doc hierarchy table

3. Wiring required — nothing works until these are done

3.1 Waitlist / contact form endpoints

Multiple pages have form inputs that submit to nowhere. Wire these to your endpoint:

<code>app/pricing/page.tsx</code>	Email waitlist input — wire to waitlist endpoint
<code>app/contact/page.tsx</code>	Contact form (name, email, org, subject, message) — wire to form handler
<code>app/page.tsx</code> (homepage)	Waitlist email input in hero section

3.2 Outcome Detail dynamic routing

`app/outcomes/[id]/page.tsx` currently hardcodes `SESSION_MARKETS` as the default. For production:

```
// Replace hardcoded data with dynamic lookup:
// import { notFound } from 'next/navigation'
// const session = await getSessionById(params.id)
// if (!session) notFound()
// return <OutcomeDetailPage session={session} />

// The file exports SESSION_MARKETS, SESSION_LS, and LSOutcomeDetailPage
// for reference — replace with real data store at launch
```

3.3 "View full session record" links

All "View full session record →" buttons on Use Case hubs currently link to `/outcomes` as a placeholder. Wire each card to its actual `/outcomes/[id]` URL at launch — this was locked as a placeholder pattern during build.

3.4 Export / download buttons

The Outcome Detail sidebar has "Download full PDF report" and "Export JSON · audit trail" buttons — these are UI only and need backend wiring.

3.5 "Run follow-on session" buttons

Follow-on session buttons appear in Outcome Detail and Use Case hub pages. These will need a route or modal to initiate a new session pre-populated with the suggested question.

4. Architecture notes per cluster

Solutions cluster

SolutionLayout.tsx and verticalData.ts live at app/solutions/ level — not in a subdirectory.

The 6 thin verticals (think-tanks, enterprise, healthcare, government, financial-services, media) import from ../verticalData using named exports.

The 5 self-contained verticals (universities, research-labs, policy, law-firms, venture-capital) have their data inline in the page file.

If moving SolutionLayout.tsx to a shared components/ directory, update all import paths from ../SolutionLayout to wherever it lands.

Use Cases cluster

UseCaseHubLayout.tsx is 'use client' because of accordion useState — all 11 hub pages inherit this.

app/use-cases/page.tsx (the index) is a server component — no 'use client' needed.

First session opens by default (defaultOpen={i === 0}). Subsequent sessions are collapsed.

hubData.ts contains all session data for 10 of 11 hubs. Universities hub has self-contained data.

Outcomes cluster

app/outcomes/page.tsx is the interactive browser — 'use client', has running/resolved toggle, category filter, WaffleGrid cards, and slide-in sidebar.

app/outcomes/hub/page.tsx is the static marketing hub — server component.

app/outcomes/[id]/page.tsx uses one template with conditional rendering on session.mode ('markets' | 'ls'). No Brier score panel renders for L&S sessions; instead shows a calibration status panel.

The PhaseTranscript component inside [id]/page.tsx uses useState for the phase tab switcher.

Index cluster

app/index/page.tsx (overview) is a server component — no interactive state needed.

app/index/explorer/page.tsx is 'use client' — useMemo for filtered results, useState for all filter controls.

app/index/heatmap/page.tsx is 'use client' — clicking a heatmap cell opens a sidebar with sessions for that domain × grade combination.

app/index/methodology/page.tsx is a server component — pure prose.

Core pages

app/page.tsx (homepage) is 'use client' — uses useEffect/setInterval for the cycling question animation and the evidence waffle panel.

app/pricing/page.tsx is 'use client' — email input uses useState. The waitlist form submits to nowhere — wire to endpoint.

app/about/page.tsx is a server component — rebuilt from v2 HTML. No team section, no milestones, no investor panel. Foundation stats (7/7/100%), IP list, contact rows.

All How it works sub-pages are server components.

5. Design system — locked values

Layout

Max width	1280px
Horizontal padding	72px desktop / 24px mobile
Nav height	64px
Nav position	Sticky top-0 z-100

Colours

sand	#E8E4D9 — page background
surface	#F4F1EB — card / section background
dark	#1E1C1A — dark sections, footer, text
terracotta	#C15F3C — primary accent
border	#C8C4BA — all 0.5px borders
muted	#B0ADA5 — secondary text
body	#5A5550 — primary body text
dark border	#49443F — border on dark bg

Typography

Headings	Libre Baskerville — serif, font-weight 400
Body	IBM Plex Sans — 19px, line-height 1.8
Labels / mono	IBM Plex Mono — 10–13px, tracking 0.06–0.1em

Logo specs

Nav logo	width="96" height="28" viewBox="0 0 487 140" fill="#1E1C1A"
Footer logo	width="120" height="34" viewBox="0 0 856 240" fill="#F7F6F2"
Footer logo – critical	Must include M567.428 (g-letter) and M842.778 (® mark) paths
Dark contexts	Use Augle_Logo_-_Dark.svg
Light contexts	Use Augle_logo-light.svg

Confidence grade colour map

Established	bg-[#D9F0E4] text-[#2A7050]
Probable	bg-[#D4E4F5] text-[#2A4A7A]
Contested	bg-[#F5E8C8] text-[#8A5A1A]
Gap	bg-[#F5D8D8] text-[#8A1818]

6. Terminology locks

These are locked — do not change during TSX implementation:

Finding	NOT "Verdict" — applies everywhere including legal docs
Confidence grade	NOT "confidence score"
Consensus	NOT "Market" (in Index context)
Corpus tiers	Gold / Silver / Flagged (website) — High/Medium/Flagged (research papers only)

Session modes	Prediction Markets / Letters & Science — no abbreviation
Guardian modes	Academic / Legal / Clinical / Financial / Editorial / Markets
Depth tiers	Rapid / Standard / Deep
Agent names	Guardian, Topic Architect, Cartographer, Methodologist, Contrarian, Synthesizer, Pragmatist

7. Pages still to build

These pages have not yet been converted to TSX. HTML files exist for some; others will need to be built from scratch.

Waitlist page

Route	/waitlist
Status	HTML not yet produced — build from design system
Notes	Email input + name + org + use case selector. Primary CTA page. Wire to waitlist endpoint.

Platform pages

Route	/platform
Status	Not yet built
Notes	Product platform overview — architecture diagram, session flow, agent breakdown

Resources / Company pages

/principles	Responsible AI — not yet built
/careers	Not yet built
/security	Security overview — not yet built
/blog	Blog index — not yet built, no posts at launch

Already have HTML for approval

<code>augle_contact.html</code>	HTML approved — TSX in <code>augle_misc_tsx.zip</code>
<code>augle_glossary.html</code>	HTML approved — TSX in <code>augle_misc_tsx.zip</code>
<code>augle_privacy.html</code>	HTML produced — awaiting approval
<code>augle_terms.html</code>	HTML produced — awaiting approval
<code>augle_terms_of_service.html</code>	HTML produced — awaiting approval

8. Known issues & decisions

Duplicate route: `/outcomes`

Two files currently claim the `/outcomes` route:

`app/outcomes/page.tsx` — the interactive browser ('use client')

`app/outcomes/hub/page.tsx` — the static marketing hub (server)

Decision needed: which one should be the primary `/outcomes` route? Recommended: make `app/outcomes/hub/page.tsx` the primary marketing landing page at `/outcomes`, and move the browser to `/outcomes/browser`. The browser is already in `augle_outcomes_index_tsx.zip` at that path.

Delaware file number — 5 legal documents

Company file number is 6988326. Five legal documents produced earlier in the project still contain the [NEW FILE NO.] placeholder. These need updating before signing:

`Augle_Bylaws.docx`

`Augle_Board_Resolutions.docx`

`Augle_SIA_Cory_Kelly.docx`

`Augle_SIA_Shubhanker_Saxena.docx`

`Augle_SIA_Steve_Dowdell.docx`

Also: 83(b) elections required for Cory and Shub within 30 days of SIA signing date.

arXiv submission

All seven research papers are published on Zenodo and SSRN. arXiv submission is pending endorsement — submit once endorsement is obtained. arXiv is valuable for visibility and prestige but not urgent for prior art (Zenodo timestamps establish that).

V3 Calibrator agent

The V3 Calibrator training is deferred until ~5,000 resolved calibration records are accumulated. No action required at launch.

Metaculus

Metaculus as a resolution source is deferred to v2. v1 resolution sources are Kalshi and Polymarket only.

9. Questions

For any questions about the TSX files, architecture decisions, or wiring:

Cory Kelly	Co-Founder · CEO · Product, design, strategy
Email	cory@augle.com
Context	All HTML files approved by Cory before TSX handoff
Shubhanker Saxena	Co-Founder · CTO · Implementation lead
Email	shub@augle.com
Claude.ai Project	Angle project in Claude.ai — full context preserved